

CMPT 225 D100 Lab 11: AVL Tree

Simon Fraser University

Overview

This is a **practice-only lab (only for attendance)**. In this lab, you will:

- Implement an AVL Tree from scratch.
- Understand the insertion and balancing strategies of an AVL Tree.

Timebox

Aim to finish in **50 minutes**.

Prerequisites

A terminal and a working C++ compiler:

```
g++ --version
```

This lab assumes the C++14 standard; feel free to swap in newer standard functionalities.

Part A: Basic Height Functions

The major difference between an AVL tree and a simple BST tree is that an AVL tree can use its nodes' height to adjust balance, avoiding the $O(n)$ worst-case performance. To balance the tree, we will first implement the functions to query a node's height if the pointer is not null, and to update a node's height given its left and right children's node pointers.

A basic definition of a struct `AVLNode` is provided to you in `AVLTree.hpp`. Since all its members are public, you are able to change any of its member variables directly. For your convenience, a simple constructor `AVLNode(int key, const T& value)` is also provided.

Inside the class `AVLTree` in `AVLTree.hpp`, please implement the following functions:

```
//Q1: Basic Height functions

int getHeight(AVLNode<T>* node) const {
    // TODO: implement this function
    // return 0 if node is null, otherwise node->height
    throw std::logic_error("not implemented!");
}
```

```
void updateHeight(AVLNode<T>* updateNode) {
    // TODO: implement this function
    throw std::logic_error("not implemented!");
}
```

Specifically, in `getHeight` you only need to check if the node pointer is null, and return 0 if null, otherwise return the node's height member variable value. In `updateHeight`, you check if the node pointer is null first, and do nothing if it is null; otherwise, call `getHeight` on the current node's left and right children pointers, and update the current node's height as the max of their return values, plus 1.

Part B: Rotations of Subtrees

AVLTree uses rotations to rebalance the tree when the tree is out of balance. In this part, you will implement the functions `rotateLeft` and `rotateRight` as member functions of the `AVLTree` class.

```
//Q2: rotations

AVLNode<T>* rotateLeft(AVLNode<T>* node) {
    // TODO: implement this function
    throw std::logic_error("not implemented!");
}
AVLNode<T>* rotateRight(AVLNode<T>* node) {
    // TODO: implement this function
    throw std::logic_error("not implemented!");
}
```

Specifically, each rotation function accepts a pointer to a subtree as its input, checks if it is null, then performs the rotation and returns the root of the rotated subtree as a result. Then, the rotation of a subtree can be performed as follows:

```
// suppose subtree is the left child of its parent
AVLNode<T>* subtreeParent = someFunction();
AVLNode<T>* subtree = subtreeParent->left;
subtree = rotateLeft(subtree);
subtreeParent->left = subtree;
```

Part C: The Insert and Balance

With rotation and height updates/queries implemented, the next last step to implement a minimal AVL tree is to implement the insert and balance features. The function `insert` is already provided to you; however, it relies on a function `insertHelper`, which you will have to implement.

```

// Q3: insert and balance

void insert(int key, const T& data) {
    this-> root = insertHelper(this-> root, key, data);
    return;
}

AVLNode<T>* insertHelper(AVLNode<T>* insertAtNode, int key, const T& data) {
    // TODO: implement this function
    throw std::logic_error("not implemented!");
}

```

The insertion of an AVL tree is similar to a BST tree; Except you need to perform an extra **restructure** step before returning to ensure the tree is balanced after insertion. Please also implement this function, using the height and rotation functions implemented previously, and use it in the `insertHelper` function. Please refer to [Lecture_Slides_Series13](#), page 18-19, for more information on restructuring and how to implement the 4 cases using rotations.

```

AVLNode<T>* restructure(AVLNode<T>* restructureAtNode) {
    // TODO: implement this function
    throw std::logic_error("not implemented!");
}

```

Additionally, you can implement printing functions in any way you like to visualize the tree structure and verify that the balancing is working after an insertion. This will help you in **Part D** testing the correctness of your implementation.

Part D: Testing

Please write your own main function, import the class `AVLTree` from `AVLTree.hpp`, and design test cases to verify the correctness of your code.